

Manipulação de Arquivos em Python

Arquivos

Em Python, o uso de arquivos nos permite **persistir** dados em disco (armazenamento secundário). Garantindo que dados se mantenham guardados entre uma execução em outra do programa.

Arquivos

“ Quem **gerencia** os arquivos é o S.O., o nosso programa python irá comunicar com o S.O. para obter acesso de leitura e escrita ao arquivo. ”

Arquivos

O que é exatamente um arquivo ?

Arquivos

“ Um arquivo é uma sequência de bytes. ”

“ O Sistema Operacional não liga para o
que há dentro do arquivo; ele apenas
enxerga bytes. ”

“ Quem dá significado aos bytes (se é uma imagem, um código Python, um PDF ou um executável) é o programa que está lendo o arquivo. ”

“ As extensões são apenas uma **dica** sobre o que se trata o arquivo e qual aplicativo deveria abrí-lo. ”

Hexdump

Podemos olhar "dentro" de arquivos utilizando a ferramenta hexdump.

Arquivos em Python

Arquivos em Python

Utilizamos a função `open()` para solicitar ao S.O. a abertura de um arquivo.

Exemplo

```
arquivo = open("nome_do_arquivo.txt", "modo_de_abertura")
```

Exemplo

```
arquivo = open("nome_do_arquivo.txt", "modo_de_abertura")
```

A função `open()` retorna uma **referência** ao arquivo.

Modos de Abertura

Modo	Descrição	OBS
r	Leitura (read)	Arquivo deve existir
w	Escrita (write)	Cria novo arquivo ou sobrescreve

Modos de Abertura

Modo	Descrição	OBS
a	Anexar (append)	Adiciona dados ao final do arquivo
x	Criação exclusiva	Falha se o arquivo já existir

Modos de Abertura

Modo	Descrição	OBS
t	Modo texto	Escreve caracteres
b	Modo binário (imgs, PDFs, ectc)	Escreve bytes

Modos de Abertura

Os modos `t` e `b` devem ser sempre combinados com algum `r`, `w`, `a` ou `x`.

```
arquivo = open("teste.txt", "w")
```

Utilizamos a variável `arquivo` para chamar funções de leitura e escrita.

Fechando Arquivos

Sempre que abrirmos um arquivo com `open()`, devemos fechá-lo com `.close()`.

```
arquivo = open("dados.txt", "r")  
  
...  
  
arquivo.close()
```

Fechando Arquivos

“ O fechamento do arquivo garante que as operações de `leitura` ou `escrita` foram concretizadas corretamente. ”

Fechando Arquivos

“ Esquecer de fechar um arquivo pode resultar em um arquivo corrompido, perda de dados, etc. ”

Escrita

```
arquivo = open("teste.txt", "w")  
arquivo.write("ABC")  
arquivo.close()
```

Append

```
arquivo = open("teste.txt", "a")  
arquivo.write("ABC")  
arquivo.close()
```

Leitura

```
arquivo = open("teste.txt", "r")  
txt = arquivo.read()  
arquivo.close()  
print(txt)
```

Gerenciador de Contexto (`with`)

A forma mais recomendada e segura de manipular arquivos em Python é utilizando o bloco `with`.

Gerenciador de Contexto

(with)

```
with open("dados.txt", "r") as arquivo:  
    conteudo = arquivo.read()
```

“ O bloco `with` garante que o arquivo seja fechado automaticamente ao final do bloco, mesmo que ocorra uma exceção. ”

Gerenciador de Contexto

(with)

```
with open("dados.txt", "r") as arquivo:  
    conteudo = arquivo.read()
```

Escrevendo em Arquivos

```
with open("saida.txt", "w") as arquivo:  
    arquivo.write("Primeira linha\n")  
    arquivo.write("Segunda linha\n")
```

Escrevendo Múltiplas Linhas

```
linhas = ["Ana\n", "João\n", "Maria\n"]  
  
with open("alunos.txt", "w") as arquivo:  
    arquivo.writelines(linhas)
```

Lendo o Conteúdo Integral

```
with open("dados.txt", "r") as arquivo:  
    conteudo = arquivo.read()  
    print(conteudo)
```

Lendo Linha por Linha

```
with open("dados.txt", "r") as arquivo:  
    linhas = arquivo.readlines()  
  
for linha in linhas:  
    print(linha)
```

Lendo uma Linha por Vez

```
with open("dados.txt", "r") as arquivo:  
    linha = arquivo.readline()  
    while linha:  
        print(linha)  
        linha = arquivo.readline()
```

Iterando sobre o Arquivo

A forma mais fácil de iterar sobre linhas:

```
with open("dados.txt", "r") as arquivo:  
    for linha in arquivo:  
        print(linha)
```

Posicionamento do Ponteiro

O Python utiliza um "ponteiro" para saber a posição atual no arquivo.

```
with open("dados.txt", "r") as arquivo:  
    print(arquivo.tell()) # Retorna a posição atual  
    conteudo = arquivo.read(10) # Lê 10 bytes  
    arquivo.seek(0) # Volta o ponteiro para o início (byte 0)
```

Encoding (Codificação de Caracteres)

É fundamental especificar a codificação para evitar problemas com acentuação (`utf-8`).

```
with open("relatorio.txt", "w", encoding="utf-8") as arquivo:  
    arquivo.write("Atenção aos acentos: á, é, í, ó, ú, ç.")
```

Manipulação de Arquivos

CSV

Para manipular dados estruturados, podemos utilizar o módulo `csv`.

```
import csv

with open("dados.csv", "w", newline="", encoding="utf-8") as file:
    writer = csv.writer(file)
    writer.writerow(["Nome", "Nota"])
    writer.writerow(["Ana", 9.5])
```

Lendo Arquivos CSV

```
import csv

with open("dados.csv", "r", encoding="utf-8") as file:
    leitor = csv.reader(file)
    for linha in leitor:
        print(linha) # Retorna uma lista por linha
```

Métodos e Atributos Úteis

```
with open("dados.txt", "r", encoding="utf-8") as f:
    print(f.name)      # Nome do arquivo: 'dados.txt'
    print(f.mode)      # Modo de abertura: 'r'
    print(f.closed)    # Verifica se está fechado: False

print(f.closed)       # Fora do bloco with: True
```